

ЛАБОРАТОРНАЯ РАБОТА № 3

«Введение в алгоритмы и структуры данных. Исследование возможностей коллекций в языке программирования Java»

Группа: РИМ-150950

Студент: Ляхова Елизавета Олеговна

Ссылка на github: https://github.com/Lizaswer/java_course_master.git

Цель: приобретение навыков работы с рекурсивными методами, знакомство с динамическими структурами данных, приобретение навыков создания и использования простейшей динамической структуры, исследование возможностей коллекций в языке программирования Java, сравнение коллекций по времени выполнения операций, знакомство с реализацией коллекций.

Ход работы:

1. Создать приложения для демонстрации примеров 1 – 5 из раздела 1. Для примера 5 дополнительно вывести последовательность обхода дерева рекурсивных вызовов.

```
package lrl3.recursion;

import java.util.ArrayList;
import java.util.List;

public class RecursionExamples {

    public static void example1(int x) {
        if (x >= 20) return;
        System.out.print(x + " ");
        example1(2 * x + 1);
    }

    public static void example2(int x) {
        if (x >= 20) return;
        example2(2 * x + 1);
        System.out.print(x + " ");
    }

    public static void example3(int x, int depth)
    {
        if (x >= 20) return;
        System.out.println("Вход: x=" + x + ",
```

```

        глубина=" + depth);
        example3(2 * x + 1, depth + 1);
        System.out.println("Выход: x=" + x + ",
        глубина=" + depth);
    }

    public static long factorial(int n) {
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }

    public static long fibonacci(int n,
    List<String> tree, String prefix) {
        tree.add(prefix + "fib(" + n + ")");
        if (n == 0) return 0;
        if (n == 1) return 1;
        long result = fibonacci(n - 1, tree,
    prefix + " ") +
        fibonacci(n - 2, tree, prefix + "
    ");
        tree.add(prefix + "-> fib(" + n + ") = "
    + result);
        return result;
    }

    public static void main(String[] args) {
        System.out.println("== Пример 1 (прямой
    порядок) ==");
    }

```

```

        example1(1);

        System.out.println("\n\n=== Пример 2
(обратный порядок) ===");
        example2(1);

        System.out.println("\n\n=== Пример 3
(глубина рекурсии) ===");
        example3(1, 1);

        System.out.println("\n=== Пример 4
(факториал) ===");

        System.out.println("5! = " +
factorial(5));

        System.out.println("10! = " +
factorial(10));

        System.out.println("\n=== Пример 5
(Фибоначчи) ===");

        List<String> tree = new ArrayList<>();
        long result = fibonacci(8, tree, "");
        System.out.println("fib(8) = " + result);
        System.out.println("\nДерево рекурсивных
ВЫЗОВОВ:");

        for (String node : tree) {
            System.out.println(node);
        }
    }
}

```

2. Создать приложение с использованием рекурсии для перевода целого числа, введенного с клавиатуры, в двоичную систему счисления.

```
package lrl3.recursion;

import java.util.Scanner;

public class DecimalToBinary {

    public static String toBinary(int n) {
        if (n == 0) return "0";
        if (n == 1) return "1";
        return toBinary(n / 2) + (n % 2);
    }

    public static void main(String[] args) {
        Scanner scanner = new
Scanner(System.in);
        System.out.print("Введите целое число:
");
        int number = scanner.nextInt();
        System.out.println("Двоичное
представление: " + toBinary(number));
        scanner.close();
    }
}
```

3. Создать приложение, позволяющее ввести и вывести одномерный массив целых чисел. Для ввода и вывода массива разработать рекурсивные методы вместо циклов for.

```
package lrl3.recursion;

import java.util.Scanner;

public class RecursiveArray {

    public static void inputArray(int[] arr, int
index, Scanner scanner) {
        if (index >= arr.length) return;
        System.out.print("arr[" + index + "] =
");
        arr[index] = scanner.nextInt();
        inputArray(arr, index + 1, scanner);
    }

    public static void printArray(int[] arr, int
index) {
        if (index >= arr.length) return;
        System.out.print(arr[index] + " ");
        printArray(arr, index + 1);
    }

    public static void main(String[] args) {
        Scanner scanner = new
Scanner(System.in);
        System.out.print("Введите размер
массива: ");
        int n = scanner.nextInt();
        int[] array = new int[n];
```

```

        System.out.println("Ввод элементов
массива:");

        inputArray(array, 0, scanner);

        System.out.print("Вывод массива: ");
        printArray(array, 0);

        scanner.close();
    }
}

```

4. Выполнить пример 1 из раздела 2. Отработать код с помощью отладчика и привести скриншоты минимум трех точек, обработанных отладчиком.

```

package lr13.linkedlist;

class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class SinglyLinkedList {

```

```
private Node head;

public void createHead(int[] values) {
    head = null;
    for (int i = values.length - 1; i >= 0;
i--) {
        Node newNode = new Node(values[i]);
        newNode.next = head;
        head = newNode;
    }
}

public void createTail(int[] values) {
    head = null;
    Node tail = null;
    for (int val : values) {
        Node newNode = new Node(val);
        if (head == null) {
            head = newNode;
            tail = newNode;
        } else {
            tail.next = newNode;
            tail = newNode;
        }
    }
}

public String toString() {
```



```
        if (head == null) return "[]";
        StringBuilder sb = new StringBuilder();
        Node current = head;
        while (current != null) {
            sb.append(current.data).append(" ");
            current = current.next;
        }
        return sb.toString().trim();
    }

    public void addFirst(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    public void addLast(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            return;
        }
        Node current = head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }
}
```

```
public void insert(int index, int data) {
    if (index < 0) return;
    if (index == 0) {
        addFirst(data);
        return;
    }
    Node current = head;
    for (int i = 0; i < index - 1 && current
!= null; i++) {
        current = current.next;
    }
    if (current == null) return;
    Node newNode = new Node(data);
    newNode.next = current.next;
    current.next = newNode;
}

public void removeFirst() {
    if (head != null) {
        head = head.next;
    }
}

public void removeLast() {
    if (head == null) return;
    if (head.next == null) {
        head = null;
        return;
    }
}
```

```

        Node current = head;
        while (current.next.next != null) {
            current = current.next;
        }
        current.next = null;
    }

    public void remove(int index) {
        if (index < 0 || head == null) return;
        if (index == 0) {
            removeFirst();
            return;
        }
        Node current = head;
        for (int i = 0; i < index - 1 &&
current.next != null; i++) {
            current = current.next;
        }
        if (current.next != null) {
            current.next = current.next.next;
        }
    }

    public void createHeadRec(int[] values, int
index) {
        if (index >= values.length) return;
        createHeadRec(values, index + 1);
        addFirst(values[index]);
    }

```

```
    }

    public void createTailRec(int[] values, int
index) {

        if (index >= values.length) return;
        addLast(values[index]);
        createTailRec(values, index + 1);
    }

    public String toStringRec() {
        return toStringRec(head);
    }

    private String toStringRec(Node node) {
        if (node == null) return "";
        return node.data + " " +
toStringRec(node.next);
    }

    public static void main(String[] args) {
        SinglyLinkedList list = new
SinglyLinkedList();

        int[] values = {10, 20, 30, 40, 50};

        System.out.println("=== Тестирование
ОДНОСВЯЗНОГО СПИСКА ===");

        list.createHead(values);
    }
}
```

```
        System.out.println("createHead: " +  
list.toString());  
  
        list.createTail(values);  
        System.out.println("createTail: " +  
list.toString());  
  
        list.addFirst(5);  
        System.out.println("addFirst(5): " +  
list.toString());  
  
        list.addLast(55);  
        System.out.println("addLast(55): " +  
list.toString());  
  
        list.insert(3, 25);  
        System.out.println("insert(3, 25): " +  
list.toString());  
  
        list.removeFirst();  
        System.out.println("removeFirst(): " +  
list.toString());  
  
        list.removeLast();  
        System.out.println("removeLast(): " +  
list.toString());  
  
        list.remove(2);  
        System.out.println("remove(2): " +
```

```

list.toString());

        System.out.println("\n=== Рекурсивные
методы ===");

        SinglyLinkedList list2 = new
SinglyLinkedList();

        list2.createHeadRec(values, 0);

        System.out.println("createHeadRec: " +
list2.toStringRec());

        SinglyLinkedList list3 = new
SinglyLinkedList();

        list3.createTailRec(values, 0);

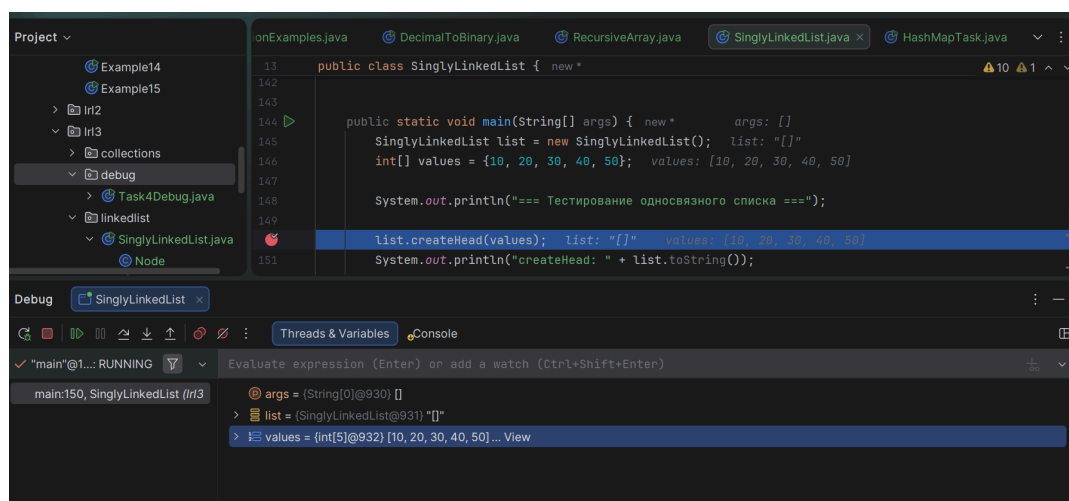
        System.out.println("createTailRec: " +
list3.toStringRec());

    }

}

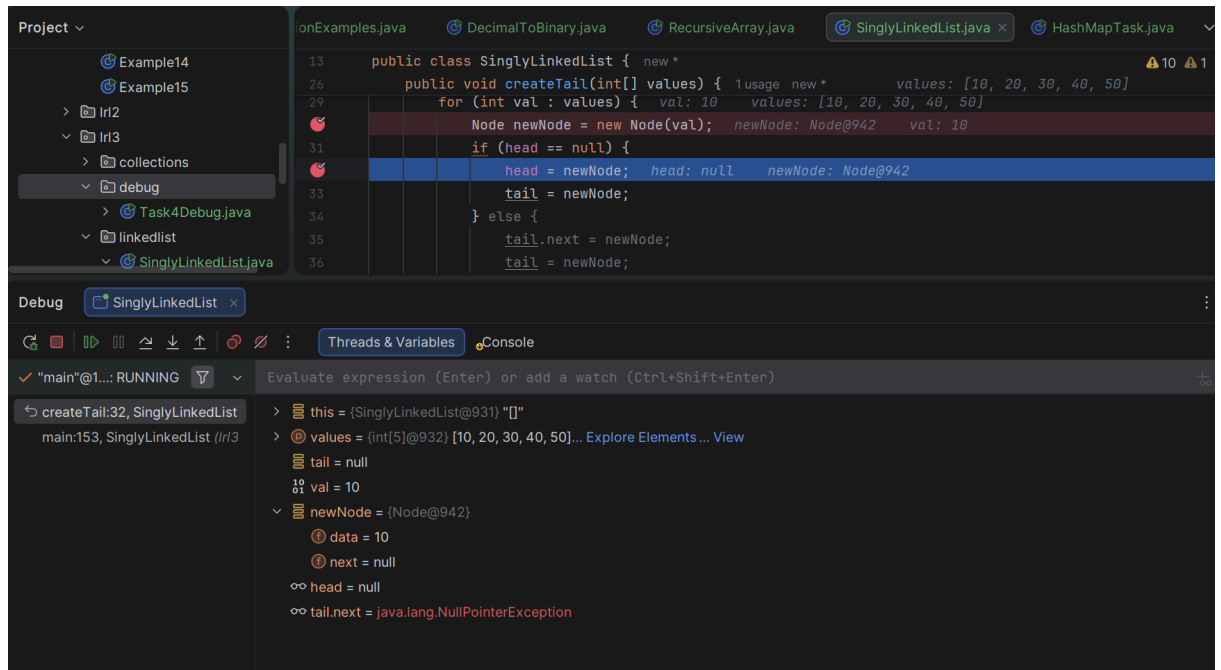
```

Скриншот 1 - Инициализация



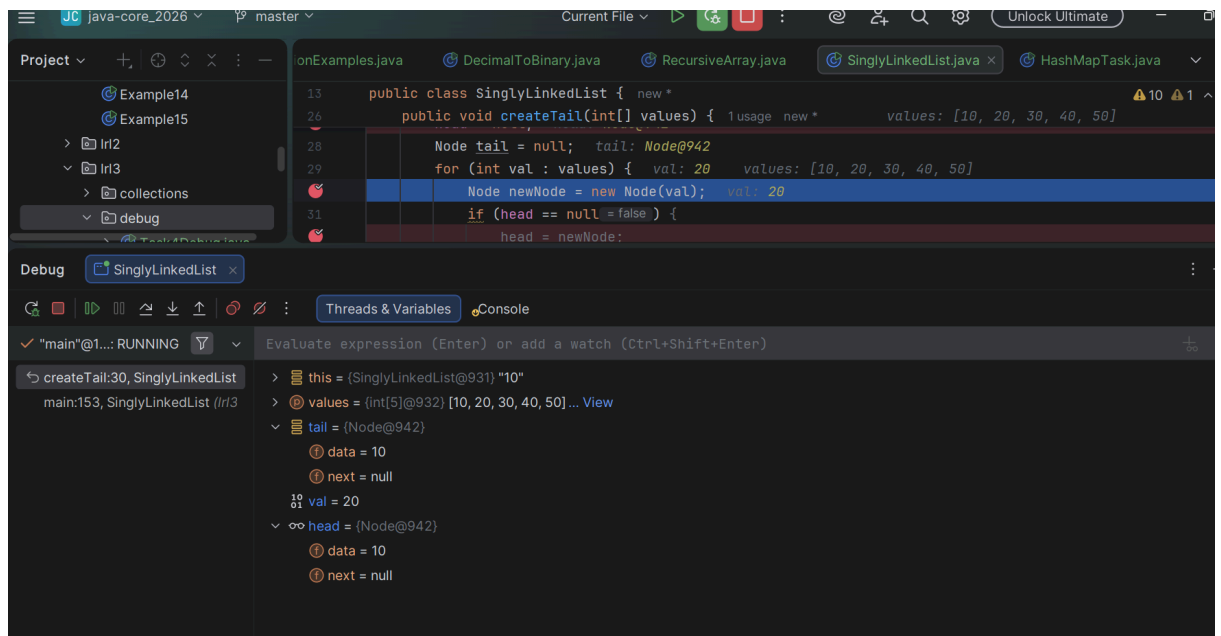
Переменные head и tail равны null, массив values содержит элементы [10, 20, 30, 40, 50].

Скриншот 2 - Создание узла



Переменная $val = 10$, создан новый узел $newNode$ с $data=10$ и $next=null$.

Скриншот 3 - Привязка головы



Теперь $head$ указывает на первый узел списка со значением 10.

5. Заполнить $HashMap$ 10 объектами $\langle Integer, String \rangle$. Найти строки у которых ключ > 5 . Если ключ $= 0$, вывести строки через запятую. Перемножить все ключи, где длина строки > 5 .

```
package lrl3.collections;

import java.util.*;

public class HashMapTask {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new
HashMap<>();

        map.put(1, "один");
        map.put(2, "два");
        map.put(3, "три");
        map.put(4, "четыре");
        map.put(5, "пять");
        map.put(6, "шесть");
        map.put(7, "семь");
        map.put(8, "восемь");
        map.put(9, "девять");
        map.put(0, "ноль");

        System.out.println("Исходный HashMap: "
+ map);

        System.out.println("\n=== Результаты
===");

        System.out.print("1. Строки с ключом >
5: ");

        List<String> result1 = new
ArrayList<>();

        for (Map.Entry<Integer, String> entry :
```



```
map.entrySet()) {  
    if (entry.getKey() > 5) {  
        result1.add(entry.getValue());  
    }  
}  
System.out.println(result1);  
  
if (map.containsKey(0)) {  
    System.out.println("2. Строка с  
ключом 0: " + map.get(0));  
}  
  
long product = 1;  
boolean found = false;  
for (Map.Entry<Integer, String> entry :  
map.entrySet()) {  
    if (entry.getValue().length() > 5) {  
        product *= entry.getKey();  
        found = true;  
        System.out.println("    Ключ " +  
entry.getKey() + " (строка: '" +  
entry.getValue() + "',  
длина: " +  
entry.getValue().length() + ")");  
    }  
}  
  
if (found) {
```

```

        System.out.println("3. Произведение
ключей (длина > 5): " + product);
    } else {
        System.out.println("3. Нет строк с
длиной > 5");
    }
}
}
}

```

6. В кругу стоят N человек, пронумерованных от 1 до N . При ведении счета по кругу вычеркивается каждый второй человек, пока не останется один. Составить две программы, моделирующие процесс. Одна из программ должна использовать класс *ArrayList*, а вторая — *LinkedList*. Какая из двух программ работает быстрее? Почему?

```

package lrl3.collections;

import java.util.*;

public class JosephusProblem {

    public static int josephusArrayList(int
        n) {
        List<Integer> list = new
            ArrayList<>();
        for (int i = 1; i <= n; i++) {
            list.add(i);
        }

        int index = 0;
        while (list.size() > 1) {

```

```

        index = (index + 1) %
list.size();
        list.remove(index);
    }
    return list.get(0);
}

public static int josephusLinkedList(int
n) {
    List<Integer> list = new
LinkedList<>();
    for (int i = 1; i <= n; i++) {
        list.add(i);
    }

    int index = 0;
    while (list.size() > 1) {
        index = (index + 1) %
list.size();
        list.remove(index);
    }
    return list.get(0);
}

public static void main(String[] args) {
    int[] sizes = {10, 100, 1000, 10000,
50000};

    System.out.println("=== Задача Иосифа
(вычеркивается каждый второй) ===\n");
    System.out.println("N\t\tArrayList
(mc)\tLinkedList (mc)");
    System.out.println("=".repeat(50));

    for (int n : sizes) {
        long start = System.nanoTime();
        int result1 =
josephusArrayList(n);

```

```

        long timeArrayList =
(System.nanoTime() - start) / 1_000_000;

        start = System.nanoTime();
        int result2 =
josephusLinkedList(n);
        long timeLinkedList =
(System.nanoTime() - start) / 1_000_000;

System.out.printf("%d\t\t%d\t\t%d\n", n,
        timeArrayList, timeLinkedList);
    }

    System.out.println("\n=== ВЫВОД
        ===");
    System.out.println("ArrayList
        работает быстрее, потому что:");
    System.out.println("1. Лучшая
        локальность данных (кэш процессора)");
    System.out.println("2. При удалении
        из ArrayList сдвиг элементов происходит в
        непрерывной памяти");
    System.out.println("3. LinkedList
        требует O(n) для доступа к элементу по
        индексу");
    }
}

```

7. Создать два проекта, в которых продемонстрировать два способа создания линейного однонаправленного списка (с головы и с хвоста) согласно примеру 2 из второго раздела. Отработать код с помощью отладчика и привести скриншоты минимум трех точек, обработанных отладчиком.

1) создание с головы

```
package lrl3.task7;
```

```
class Node {  
    int data;  
    Node next;
```

```
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

```
public class CreateHeadDemo {  
    private Node head;
```

```
    // Создание списка с головы (пример 2 из раздела 2)
```

```
    public void createHead(int[] values) {
```

```
        head = null; // ТОЧКА 1
```

```
        for (int i = values.length - 1; i >= 0; i--) {
```

```
            Node newNode = new Node(values[i]); // ТОЧКА 2
```

```
            newNode.next = head; // ТОЧКА 3
```

```
            head = newNode; // ТОЧКА 3
```

```
        }
```

```
    }
```

```
    public void printList() {
```

```
        Node current = head;
```

```
        while (current != null) {
```

```
            System.out.print(current.data + " ");
```

```
            current = current.next;
```

```
        }
```

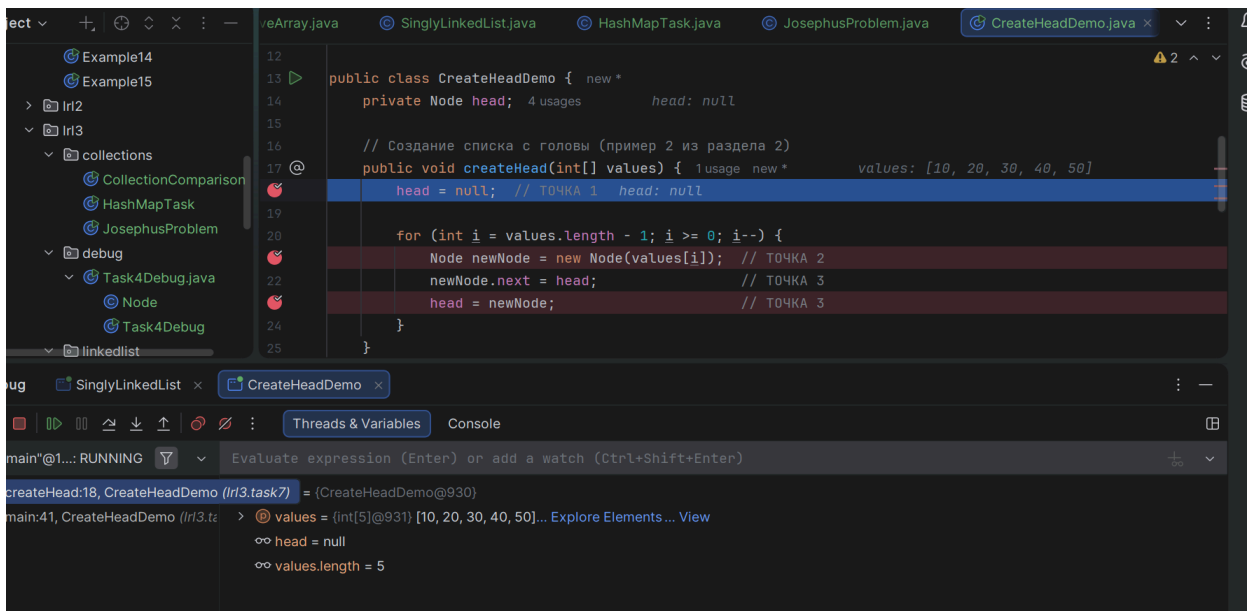
```
        System.out.println();
```

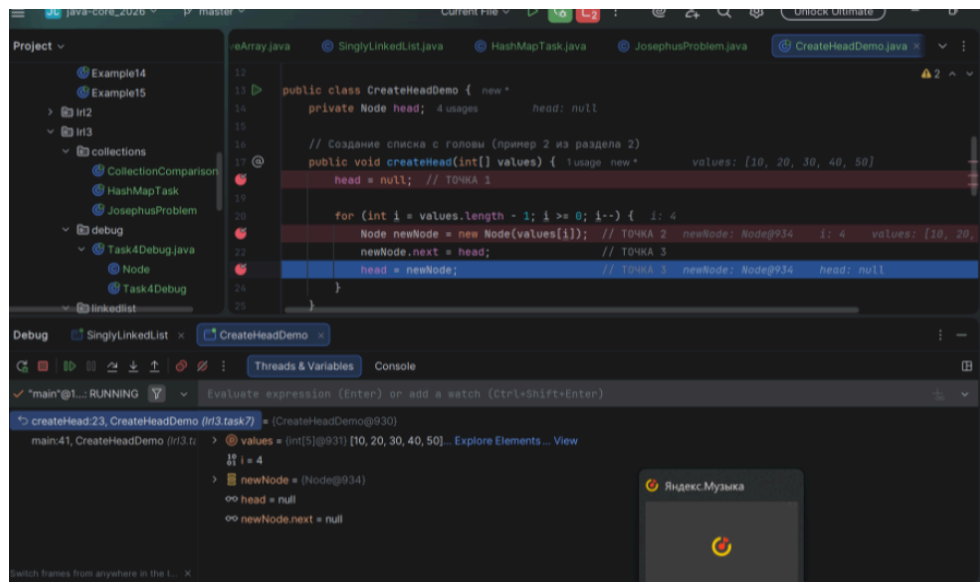
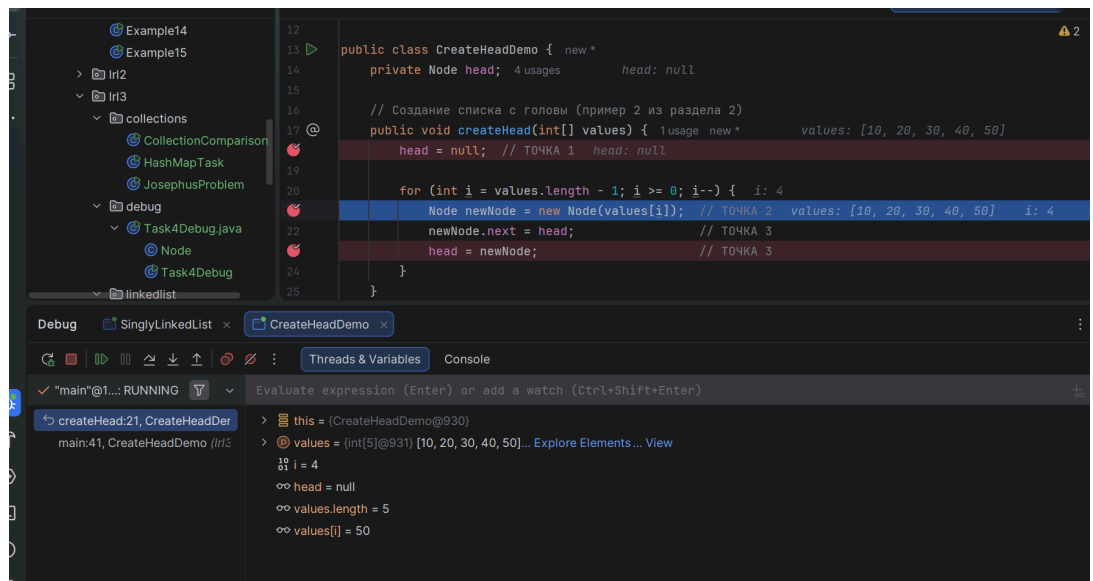
```
    }
```

```
    public static void main(String[] args) {
```

```
CreateHeadDemo list = new CreateHeadDemo();  
int[] values = {10, 20, 30, 40, 50};
```

```
System.out.println("=== Создание списка С ГОЛОВЫ  
===");  
list.createHead(values);  
System.out.print("Результат: ");  
list.printList();  
}  
}
```





2) создание с хвоста

package lrl3.task7;

```

class Node {
    int data;
    Node next;

```

```

Node(int data) {
    this.data = data;

```

```

        this.next = null;
    }
}

public class CreateTailDemo {
    private Node head;

    // Создание списка с хвоста (пример 2 из раздела 2)
    public void createTail(int[] values) {
        head = null;    // ТОЧКА 1
        Node tail = null; // ТОЧКА 1

        for (int val : values) {
            Node newNode = new Node(val); // ТОЧКА 2

            if (head == null) {
                head = newNode;    // ТОЧКА 3
                tail = newNode;    // ТОЧКА 3
            } else {
                tail.next = newNode;
                tail = newNode;
            }
        }
    }

    public void printList() {
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        CreateTailDemo list = new CreateTailDemo();
        int[] values = {10, 20, 30, 40, 50};
    }
}

```



```

        System.out.println("=== Создание списка С ХВОСТА
===");
        list.createTail(values);
        System.out.print("Результат: ");
        list.printList();
    }
}

```

8. Разработать проект, в котором для ввода, вывода и изменения односвязного линейного списка создать следующие методы:

а) с использованием цикла:

- ввод с головы *createHead()*;
- ввод с хвоста *createTail()*;
- вывод (возвращается строка, сформированная из элементов списка) *toString()*;

- добавление элемента в начало списка *addFirst()*;
- добавление элемента в конец списка *addLast()*;
- вставка элемента в список с указанным номером *insert()*;
- удаление элемента с головы списка *removeFirst()*;
- удаление последнего элемента списка *removeLast()*;
- удаление из списка элемента с указанным номером *remove()*;

б) с использованием рекурсии:

- ввод с головы *createHeadRec()*;
- ввод с хвоста *createTailRec()*;
- вывод (возвращается строка, сформированная из элементов

cnucka) toStringRec().

```
package lrl3.linkedlist;

class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class SinglyLinkedList {
    private Node head;

    public void createHead(int[] values) {
        head = null;
        for (int i = values.length - 1; i >= 0; i--)
        {
            Node newNode = new Node(values[i]);
            newNode.next = head;
            head = newNode;
        }
    }

    public void createTail(int[] values) {
        head = null;
        Node tail = null;
        for (int val : values) {
            Node newNode = new Node(val);
            if (head == null) {
                head = newNode;
                tail = newNode;
            }
        }
    }
}
```

```

        } else {
            tail.next = newNode;
            tail = newNode;
        }
    }
}

public String toString() {
    if (head == null) return "[]";
    StringBuilder sb = new StringBuilder();
    Node current = head;
    while (current != null) {
        sb.append(current.data).append(" ");
        current = current.next;
    }
    return sb.toString().trim();
}

public void addFirst(int data) {
    Node newNode = new Node(data);
    newNode.next = head;
    head = newNode;
}

public void addLast(int data) {
    Node newNode = new Node(data);
    if (head == null) {
        head = newNode;
        return;
    }
    Node current = head;
    while (current.next != null) {
        current = current.next;
    }
    current.next = newNode;
}

public void insert(int index, int data) {

```

```

        if (index < 0) return;
        if (index == 0) {
            addFirst(data);
            return;
        }
        Node current = head;
        for (int i = 0; i < index - 1 && current !=
null; i++) {
            current = current.next;
        }
        if (current == null) return;
        Node newNode = new Node(data);
        newNode.next = current.next;
        current.next = newNode;
    }

    public void removeFirst() {
        if (head != null) {
            head = head.next;
        }
    }

    public void removeLast() {
        if (head == null) return;
        if (head.next == null) {
            head = null;
            return;
        }
        Node current = head;
        while (current.next.next != null) {
            current = current.next;
        }
        current.next = null;
    }

    public void remove(int index) {
        if (index < 0 || head == null) return;
        if (index == 0) {

```

```

        removeFirst();
        return;
    }
    Node current = head;
    for (int i = 0; i < index - 1 && current.next
!= null; i++) {
        current = current.next;
    }
    if (current.next != null) {
        current.next = current.next.next;
    }
}

    public void createHeadRec(int[] values, int
index) {
        if (index >= values.length) return;
        createHeadRec(values, index + 1);
        addFirst(values[index]);
    }

    public void createTailRec(int[] values, int
index) {
        if (index >= values.length) return;
        addLast(values[index]);
        createTailRec(values, index + 1);
    }

    public String toStringRec() {
        return toStringRec(head);
    }

    private String toStringRec(Node node) {
        if (node == null) return "";
        return node.data + " " +
toStringRec(node.next);
    }

```

```
public static void main(String[] args) {
    SinglyLinkedList list = new
SinglyLinkedList();
    int[] values = {10, 20, 30, 40, 50};

    System.out.println("=== Тестирование
односвязного списка ===");

    list.createHead(values);
    System.out.println("createHead: " +
list.toString());

    list.createTail(values);
    System.out.println("createTail: " +
list.toString());

    list.addFirst(5);
    System.out.println("addFirst(5): " +
list.toString());

    list.addLast(55);
    System.out.println("addLast(55): " +
list.toString());

    list.insert(3, 25);
    System.out.println("insert(3, 25): " +
list.toString());

    list.removeFirst();
    System.out.println("removeFirst(): " +
list.toString());

    list.removeLast();
    System.out.println("removeLast(): " +
list.toString());

    list.remove(2);
```

```

        System.out.println("remove(2): " +
list.toString());

        System.out.println("\n=== Рекурсивные методы
===");
        SinglyLinkedList list2 = new
SinglyLinkedList();
        list2.createHeadRec(values, 0);
        System.out.println("createHeadRec: " +
list2.toStringRec());

        SinglyLinkedList list3 = new
SinglyLinkedList();
        list3.createTailRec(values, 0);
        System.out.println("createTailRec: " +
list3.toStringRec());
    }

```

Вопросы:

В чем разница между fail-safe и fail-fast свойствами?

Fail-fast итераторы (ArrayList, HashMap, HashSet) немедленно выбрасывают `ConcurrentModificationException`, если коллекция изменяется после создания итератора (кроме метода `remove()` самого итератора). Они используют счетчик модификаций (`modCount`), который проверяется перед каждой операцией. Если коллекция была изменена другим способом, итератор "быстро проваливается" с исключением.

Fail-safe итераторы (CopyOnWriteArrayList, ConcurrentHashMap) работают с копией (снимком) данных на момент создания итератора. Они не выбрасывают исключений при конкурентных изменениях, но могут

не видеть свежих изменений, сделанных после создания итератора.

Недостаток: требуют больше памяти из-за создания копии.

Основные отличия:

Fail-fast: выбрасывает исключение при конкурентных изменениях

Fail-safe: не выбрасывает исключение, работает с копией данных

В чем разница между Iterator и Enumeration?

Iterator и Enumeration — это два интерфейса в Java, предназначенные для перебора элементов коллекций, но они имеют существенные различия.

Основные отличия:

- 1. Время появления: Enumeration появился в Java 1.0 и считается устаревшим, а Iterator появился в Java 1.2 и является современным стандартом.*
- 2. Метод удаления: Iterator имеет метод `remove()`, который позволяет удалять элементы из коллекции во время обхода. Enumeration не поддерживает удаление элементов.*
- 3. Названия методов:*
 - Enumeration использует методы `hasMoreElements()` и `nextElement()`*
 - Iterator использует более короткие и удобные методы `hasNext()` и `next()`*
- 4. Поддержка коллекций: Iterator поддерживается всеми современными коллекциями Java (ArrayList, HashSet, HashMap и др.).*

Enumeration поддерживается только устаревшими классами, такими как `Vector` и `Hashtable`.

5. *Безопасность: `Iterator` является fail-fast (быстро завершается), то есть выбрасывает исключение `ConcurrentModificationException`, если коллекция была изменена во время обхода. `Enumeration` не имеет такого механизма.*

Рекомендация: Всегда следует использовать `Iterator` вместо `Enumeration`, так как он предоставляет больше возможностей (удаление элементов) и является более безопасным.

В чем разница между `Iterator` и `ListIterator`?

`Iterator` и `ListIterator` — это интерфейсы для перебора элементов, где `ListIterator` является расширенной версией `Iterator`.

Основные отличия:

1. *Применение: `Iterator` может использоваться для перебора элементов любой коллекции, реализующей интерфейс `Collection` (`List`, `Set`, `Queue`). `ListIterator` работает только с коллекциями, реализующими интерфейс `List` (`ArrayList`, `LinkedList` и др.).*
2. *Направление обхода: `Iterator` поддерживает только однонаправленный обход — только вперед. `ListIterator` поддерживает двунаправленный обход — как вперед, так и назад (методы `hasPrevious()` и `previous()`).*

3. *Получение индекса: Iterator не предоставляет возможности получить индекс текущего элемента. ListIterator позволяет получить индекс текущего элемента с помощью методов `nextIndex()` и `previousIndex()`.*
4. *Добавление элементов: Iterator не имеет метода для добавления элементов в коллекцию. ListIterator имеет метод `add()`, который позволяет вставлять новые элементы в текущую позицию.*
5. *Изменение элементов: Iterator не позволяет изменять существующие элементы. ListIterator имеет метод `set()`, который позволяет заменить текущий элемент новым значением.*
6. *Позиция курсора: Iterator всегда указывает на позицию между элементами. ListIterator предоставляет более точный контроль над позицией курсора.*

Когда что использовать:

Используйте Iterator, если нужен простой однонаправленный обход любой коллекции

Используйте ListIterator, если работаете со списком и нужен двунаправленный обход, доступ к индексам, добавление или изменение элементов

Дайте определение понятию “итератор”.

Итератор — это объект, реализующий паттерн проектирования "Итератор", который предоставляет унифицированный способ последовательного доступа к элементам коллекции без раскрытия её внутренней структуры. Итератор позволяет обходить коллекцию,

получать следующий элемент и (в некоторых реализациях) удалять элементы во время обхода.

Основные методы итератора:

`hasNext()` — проверяет, есть ли следующий элемент

`next()` — возвращает следующий элемент и перемещает курсор

`remove()` — удаляет текущий элемент

Как задается порядок следования объектов в коллекции, как отсортировать коллекцию?

Способы задания порядка:

Natural ordering (естественный порядок) — реализация интерфейса `Comparable` с методом `compareTo()`. Используется для таких классов, как `String`, `Integer`, `Date`.

Custom ordering (пользовательский порядок) — реализация интерфейса `Comparator` с методом `compare()`. Позволяет задать произвольный порядок сортировки без изменения самого класса.

```
Collections.sort(list);           // естественный порядок
```

```
Collections.sort(list, comparator); // пользовательский порядок
```

```
list.sort(Comparator.naturalOrder()); // Java 8+
```

```
list.sort((a, b) -> a.compareTo(b)); // лямбда-выражение
```

Как получить коллекцию только для чтения?

```
List<String> safeReadOnly = Collections.unmodifiableList(new  
ArrayList<>(original));
```

Как получить не модифицируемую коллекцию?

// Java 9+

```
List<String> list = List.of("a", "b", "c");  
Set<String> set = Set.of("a", "b", "c");  
Map<String, String> map = Map.of("key", "value");
```

// Java 10+ (с var)

```
var immutableList = List.of("a", "b", "c");
```

// Через копирование

```
List<String> copy = List.copyOf(originalList);
```

Как получить синхронизированную коллекцию из не синхронизированной?

// Обычные коллекции (не синхронизированные)

```
List<String> list = new ArrayList<>();
```

```
Set<String> set = new HashSet<>();
```

```
Map<String, String> map = new HashMap<>();
```

// Синхронизированные обертки

```
List<String> syncList = Collections.synchronizedList(list);
```

```
Set<String> syncSet = Collections.synchronizedSet(set);
```

```
Map<String, String> syncMap = Collections.synchronizedMap(map);
```

Как реализован цикл foreach?

// Исходный код

```
for (String s : list) {  
    System.out.println(s);  
}
```

// После компиляции (примерно)

```
for (Iterator<String> it = list.iterator(); it.hasNext();) {  
    String s = it.next();  
    System.out.println(s);  
}
```

Какие есть способы перебора всех элементов List?

с помощью цикла

```
for (int i = 0; i < list.size(); i++) {  
    System.out.println(list.get(i));  
}
```

с помощью foreach

```
for (String item : list) {  
    System.out.println(item);  
}
```

с помощью итератора

```
Iterator<String> it = list.iterator();  
while (it.hasNext()) {  
    System.out.println(it.next());  
}
```

```
}  
  
с помощью ListIterator  
ListIterator<String> lit = list.listIterator();  
while (lit.hasNext()) {  
    System.out.println(lit.next());  
}
```

Какие коллекции синхронизированы?

Устаревшие синхронизированные коллекции (Java 1.0-1.1):

- `Vector` — синхронизированный аналог `ArrayList`
- `Stack` — наследуется от `Vector`
- `Hashtable` — синхронизированный аналог `HashMap`

Современные синхронизированные коллекции:

- `Collections.synchronizedXxx()` — обертки для любых коллекций
- `ConcurrentHashMap` — высокопроизводительная синхронизация для `Map`
- `CopyOnWriteArrayList` — для сценариев "много чтения, мало записи"
- `CopyOnWriteArraySet`
- `BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`)

Важно: Даже синхронизированные коллекции требуют дополнительной синхронизации при итерации.

Какую функциональность представляет класс `Collections`?

Класс `Collections` — это утилитарный класс с статическими методами для работы с коллекциями:

Сортировка и поиск:

- `sort()` — сортировка списка
- `binarySearch()` — бинарный поиск
- `reverse()` — разворот списка
- `shuffle()` — перемешивание элементов

Поиск экстремумов:

- `min()` / `max()` — минимальный/максимальный элемент

Создание оберток:

- `synchronizedXxx()` — синхронизированные обертки
- `unmodifiableXxx()` — неизменяемые обертки
- `checkedXxx()` — типобезопасные обертки

Заполнение и копирование:

- `fill()` — заполнение коллекции
- `copy()` — копирование элементов
- `addAll()` — добавление нескольких элементов

Создание пустых коллекций:

- `emptyList()`, `emptySet()`, `emptyMap()`

Одноэлементные коллекции:

- `singletonList()`, `singleton()`, `singletonMap()`

Частотность:

`frequency()` — подсчет вхождений элемента

`disjoint()` — проверка на отсутствие общих элементов

Почему Map не наследуется от Collection?

Основные причины:

1. Разная структура хранения: *Collection* хранит одиночные элементы, *Map* хранит пары "ключ-значение".
2. Разные методы: У *Map* нет методов `add()`, `remove(Object)`, `iterator()` возвращает не просто элементы, а `Set<Entry>`.
3. Логическая несовместимость: Нельзя единообразно работать с *Collection* и *Map*. Что должен возвращать метод `size()`? Количество ключей или количество пар?
4. Принцип разделения интерфейсов (ISP): Лучше иметь два независимых интерфейса, чем один перегруженный.

Связь через методы:

`keySet()` возвращает `Set (Collection)`

`values()` возвращает `Collection`

`entrySet()` возвращает `Set (Collection)`

Почему в классе *iterator* нет метода для получения следующего элемента без передвижения курсора?

Причины:

Принцип единственной ответственности: Итератор спроектирован для последовательного однонаправленного обхода. Каждый вызов `next()` перемещает курсор.

Простота реализации: Отсутствие метода "посмотреть без перемещения" упрощает реализацию итераторов для всех типов коллекций.

Безопасность: Если бы был метод для просмотра без перемещения, это могло бы привести к путанице и ошибкам (например, бесконечные циклы).

Альтернативы: Если нужен доступ без перемещения, можно сохранить элемент в переменной или использовать `ListIterator` для списков.

Почему нет метода `iterator.add()` чтобы добавить элементы в коллекцию?

Причины:

Нарушение контракта итератора: Основная цель итератора — обход элементов, а не их добавление. Добавление может нарушить позицию курсора.

Неоднозначность позиции добавления: Куда добавлять? До текущего элемента или после? Для разных коллекций это может означать разное.

Для списков есть `ListIterator`: У `ListIterator` есть метод `add()`, который добавляет элемент в текущую позицию.

Разные требования для разных коллекций: Для Set добавление может быть неоднозначным (порядок не определен). Для очередей добавление происходит в конец.

Чистота интерфейса: Лучше иметь специализированные интерфейсы (`ListIterator`) для специфических операций.

Расскажите о реализации классов `HashSet` и `TreeSet`?

HashSet:

- Основан на `HashMap<E, Object>`
- Элементы хранятся как ключи, `value` = константа `PRESENT` (dummy object)
- Использует хеш-таблицу (массив + связанные списки/деревья при коллизиях)
- Время операций: $O(1)$ в среднем, $O(n)$ в худшем
- Неупорядоченный (порядок не гарантирован)
- Допускает один null элемент
- Начальная емкость 16, коэффициент загрузки 0.75

TreeSet:

Основан на `TreeMap<E, Object>`

Реализует красно-черное дерево (самобалансирующееся бинарное дерево поиска)

Элементы хранятся в отсортированном порядке

Время операций: $O(\log n)$ для `add`, `remove`, `contains`

Не допускает `null` (Java 7+)

Поддерживает навигационные методы: `first()`, `last()`, `lower()`, `higher()`, `floor()`, `ceiling()`, `subSet()`, `headSet()`, `tailSet()`

Чем отличаются `ArrayDeque` и `TreeMap`? Как они устроены и работают? Что со временем доступа к объектам, какие зависимости?

`ArrayDeque`:

- Реализует интерфейс `Deque` (двусторонняя очередь)
- Основан на массиве с циклическими указателями (ring buffer)
- Динамически расширяется при заполнении
- Операции: $O(1)$ для `addFirst`, `addLast`, `removeFirst`, `removeLast`
- Нет доступа по индексу
- Хранит только элементы (не пары)

`TreeMap`:

- Реализует интерфейс `NavigableMap` (отсортированная карта)
- Основан на красно-черном дереве
- Хранит пары ключ-значение
- Операции: $O(\log n)$ для `put`, `get`, `remove`
- Отсортирован по ключам
- Поддерживает навигационные методы

Что будет, если в Map положить два значения с одинаковым ключом?

Старое значение будет заменено новым. Метод `put()` возвращает старое значение (или `null`, если ключа не было).

```
Map<String, String> map = new HashMap<>();  
map.put("key", "value1");  
String old = map.put("key", "value2");  
System.out.println(old); // "value1"  
System.out.println(map.get("key")); // "value2"
```

Что делать, чтобы не возникло исключение `ConcurrentModificationException`?

1. *Использовать fail-safe коллекции:*

```
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
```

2. *Использовать итератор для удаления:*

```
Iterator<String> it = list.iterator();  
  
while (it.hasNext()) {  
  
    if (condition) {  
  
        it.remove(); // правильно  
  
        // list.remove(element); // вызовет исключение  
  
    }  
  
}
```

Что такое Hashtable, чем она отличается от HashMap? На сегодняшний день она deprecated, как все-таки использовать нужную функциональность?

Hashtable:

- Устаревший класс из Java 1.0
- Синхронизированный (потокобезопасный)
- Не допускает null ключи и null значения
- Медленнее из-за синхронизации на уровне методов

Вывод: приобрели навыки работы с рекурсивными методами, познакомились с динамическими структурами данных, приобрели навыки создания и использования простейшей динамической структуры, исследовали возможности коллекций в языке программирования Java, сравнили коллекции по времени выполнения операций, познакомились с реализацией коллекций.